

Building Fullstack Apps with Cursor

Maurice de Beijer
@mauricedb



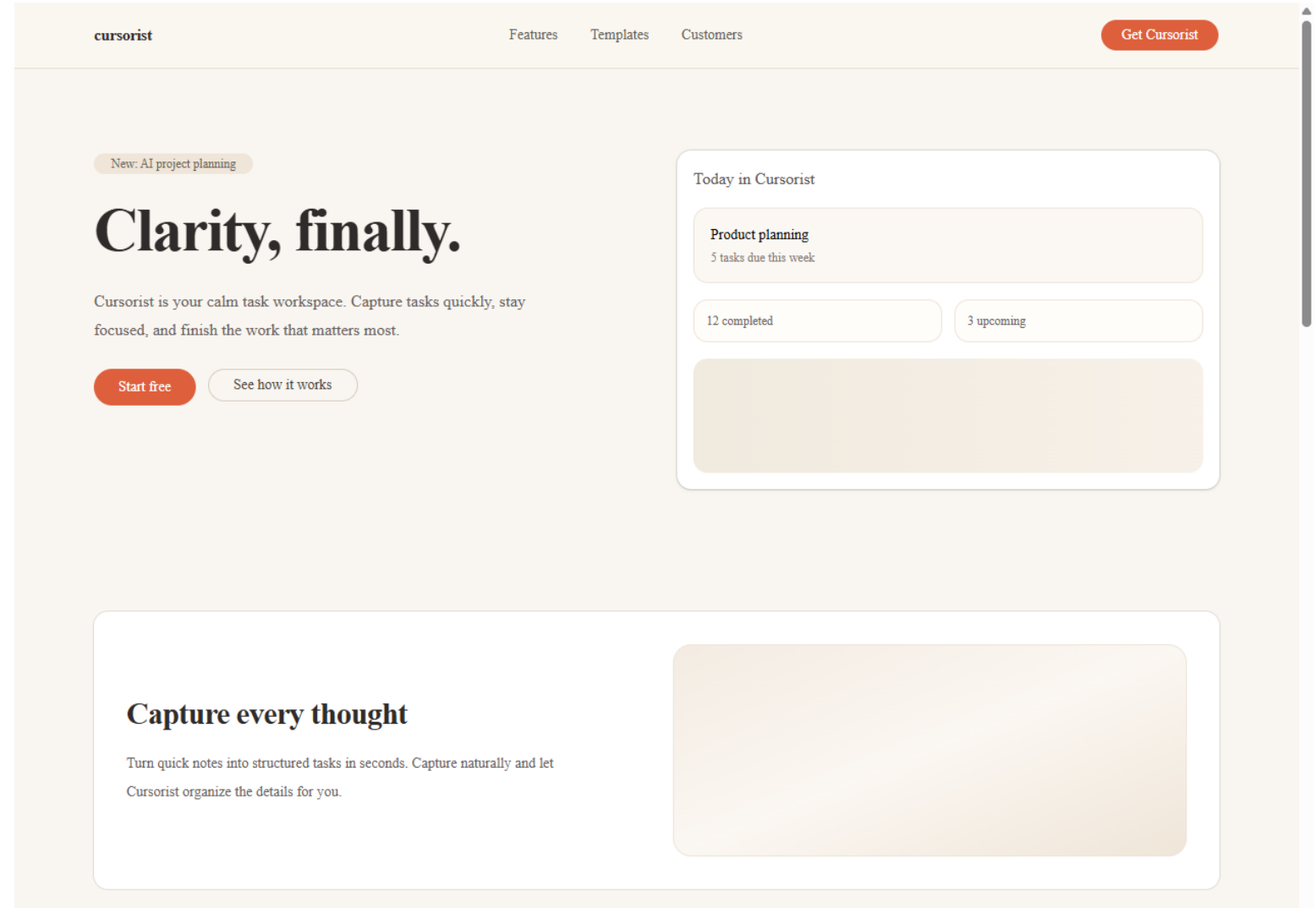
- Maurice de Beijer
- The Problem Solver
- Freelance developer/instructor
- Twitter: [@mauricedb](https://twitter.com/mauricedb)
- Web: <http://www.theproblemsolver.dev/>
- E-mail: maurice.de.beijer@gmail.com



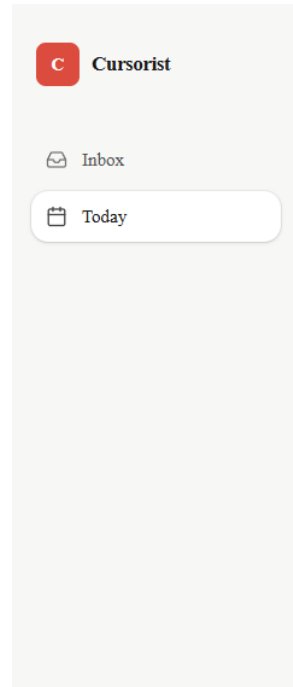
What We'll Build Today

- **What we're building:**
 - A full stack Task Management app (Cursorist)
 - Landing page, inbox view, today view
 - Real database with Prisma + SQLite
 - “Polished” UI with Shadcn components
- **What you'll master:**
 - **AI orchestration with Cursor Composer** — multi-file changes from a single prompt
 - **The Cursor Rules framework** — enforce standards and prevent AI drift
 - **Full stack scaffolding** — type-safe schemas and server actions in minutes
 - **High-fidelity UI iteration** — from prompt to polished interface

The landing
page we will
build



The task management pages we will build



Today

Sat, May 2

☐ Write product requirements document
Apr 25, 2026

☐ Review pull requests
May 2, 2026

☐ Write project status update
May 2, 2026

Write a new task...



Add Cancel

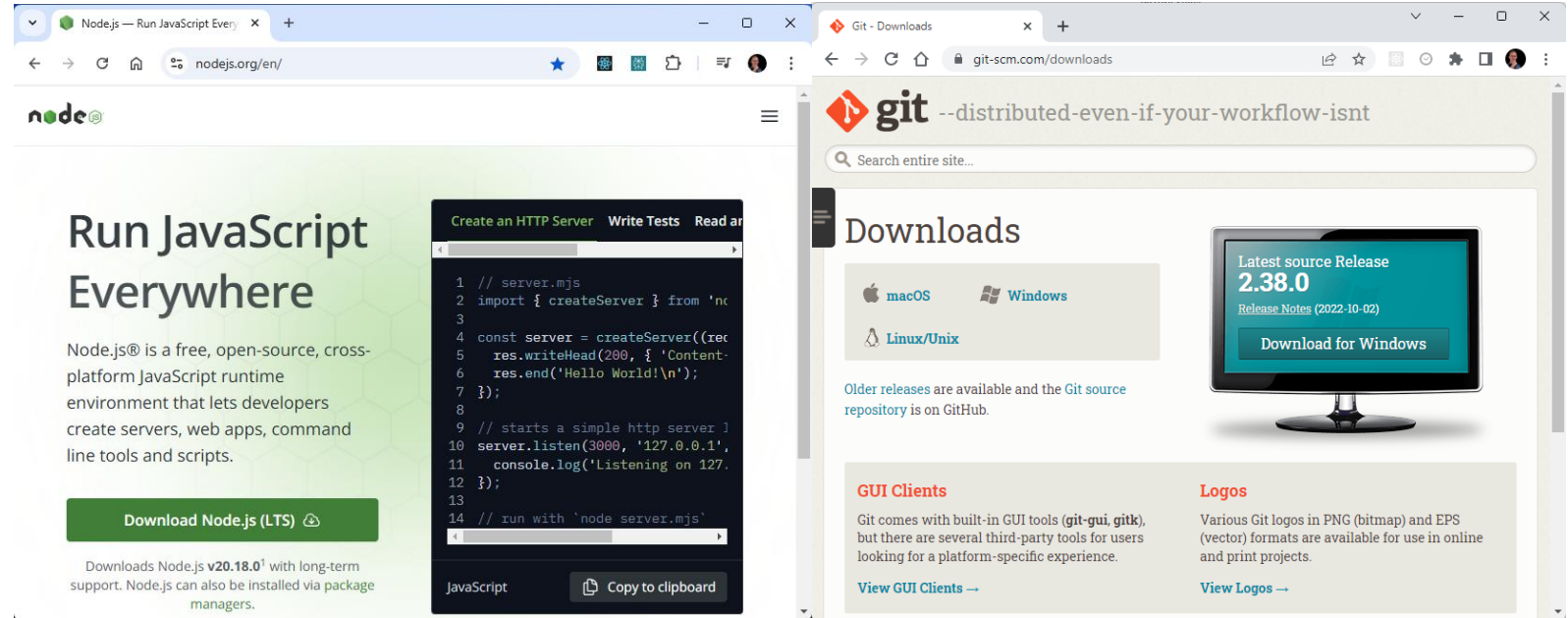
Prerequisites

Install Node & NPM

Install the GitHub repository

Install Cursor

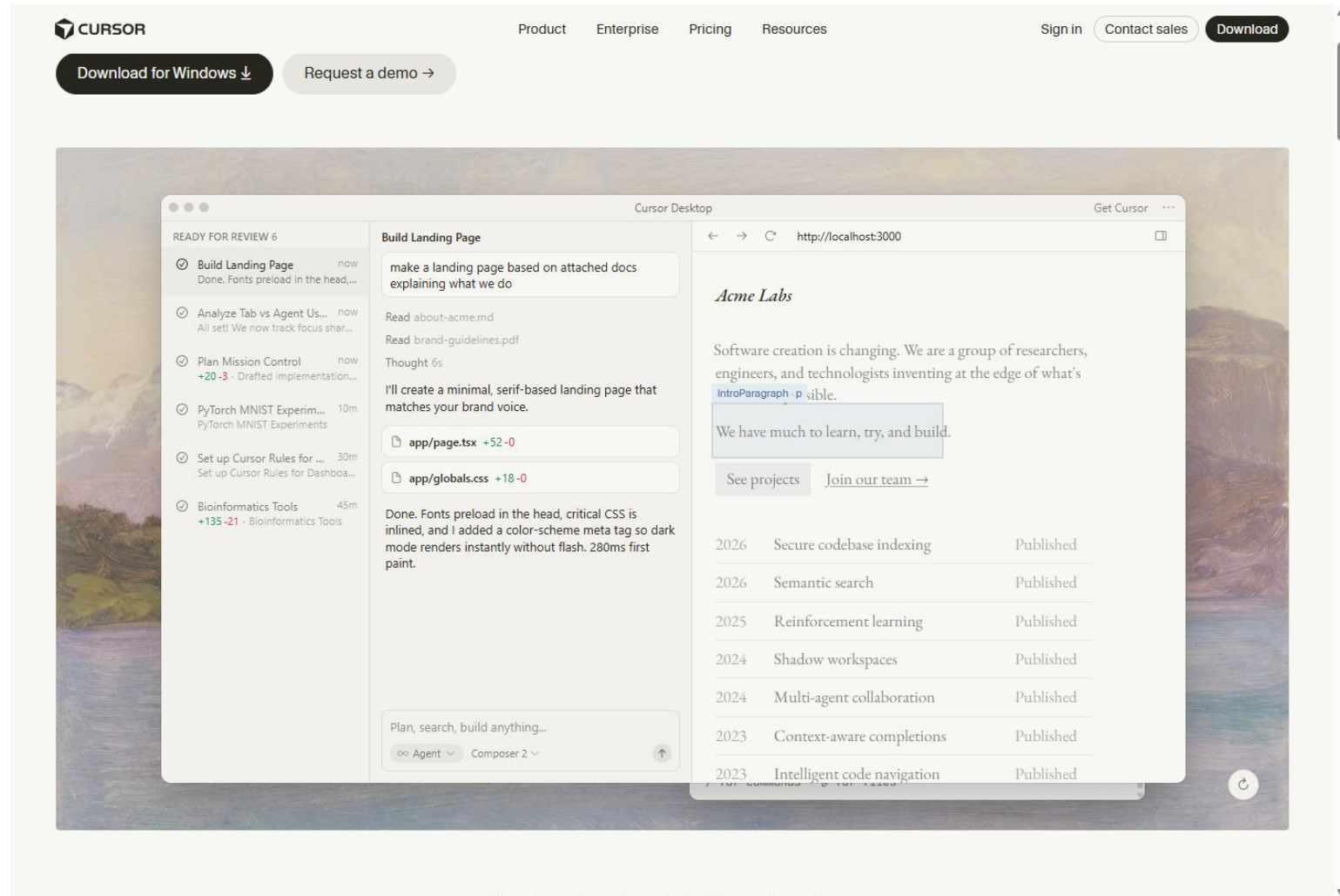
Install Node.js & NPM



```
Windows PowerShell

PS C:\demos> node --version
v20.9.0
PS C:\demos> git --version
git version 2.45.2.windows.1
PS C:\demos> npm --version
10.2.4
PS C:\demos> |
```

Install Cursor



Following Along



- Repo with start code: <https://github.com/mauricedb/cursorist>
- Workshop resources: <https://github.com/mauricedb/cursorist-workshop>

Install the start Repository

```
next-server (v16.2.4) x + v
PS C:\demos> git clone https://github.com/mauricedb/cursorist.git
Cloning into 'cursorist'...
remote: Enumerating objects: 194, done.
remote: Counting objects: 100% (194/194), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 194 (delta 62), reused 189 (delta 57), pack-reused 0 (from 0)
Receiving objects: 100% (194/194), 279.77 KiB | 3.04 MiB/s, done.
Resolving deltas: 100% (62/62), done.
PS C:\demos> cd .\cursorist\
PS C:\demos\cursorist> npm install
npm WARN deprecated prebuild-install@7.1.3: No longer maintained. Please contact the author of the relevant native addon; alternatives are available.

added 481 packages, and audited 482 packages in 26s

164 packages are looking for funding
  run `npm fund` for details

5 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

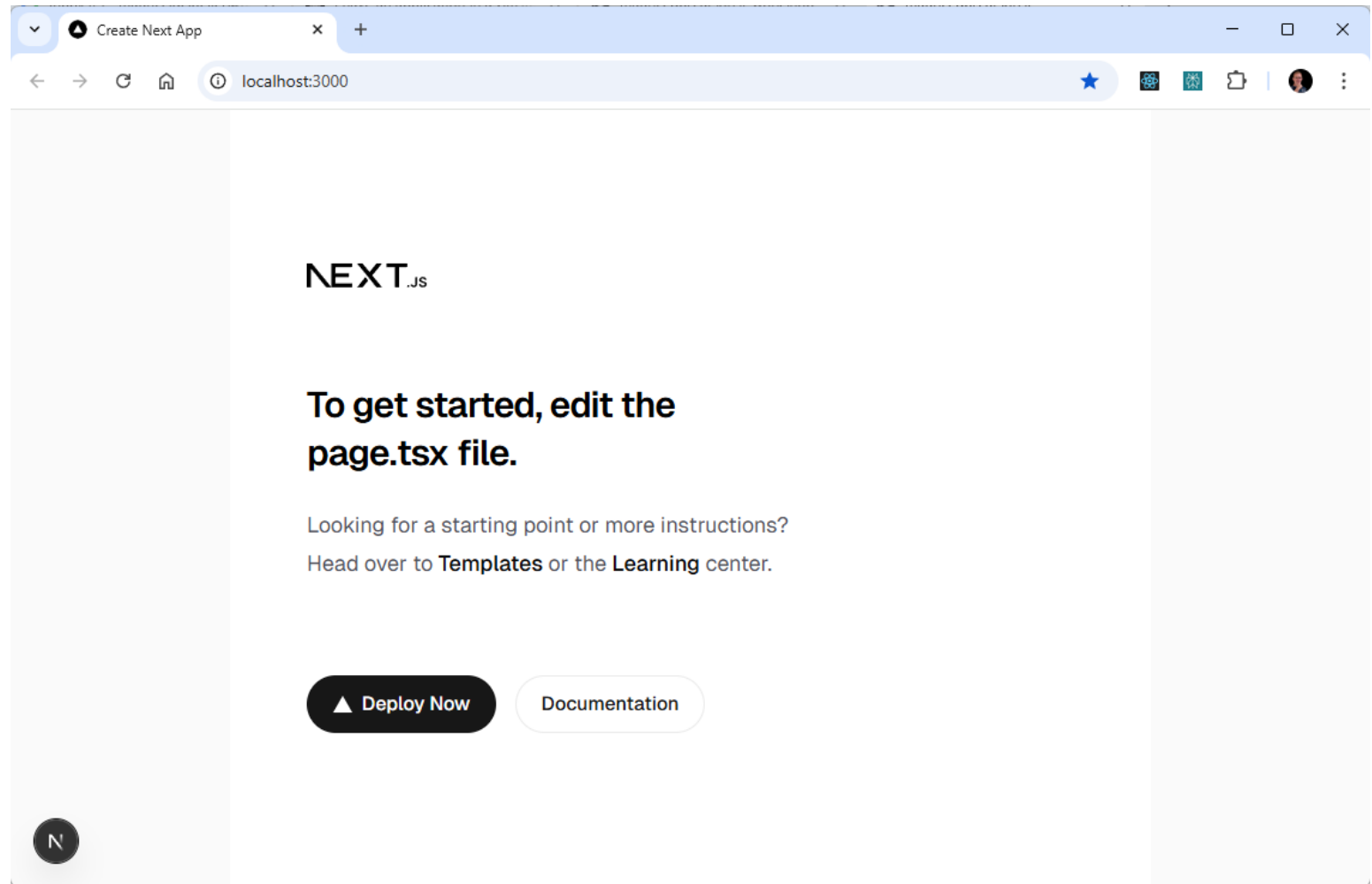
Run `npm audit` for details.
PS C:\demos\cursorist> npm run dev

> cursorist@0.1.0 dev
> next dev

▲ Next.js 16.2.4 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.144.1:3000
- Environments: .env
✓ Ready in 489ms

GET / 200 in 1692ms (next.js: 1532ms, application-code: 160ms)
GET / 200 in 59ms (next.js: 13ms, application-code: 46ms)
[browser] Image with src "http://localhost:3000/vercel.svg" has either width or height modified, but not the other. If you use CSS to change the size of your image, also include the styles 'width: "auto"' or 'height: "auto"' to maintain the aspect ratio.
```

The application



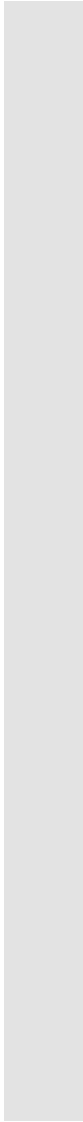
What the starter repo already includes

- A new Next.js application with the default settings
 - `npx create-next-app@latest`
- Prisma with SQLite support
 - `npx prisma init --datasource-provider sqlite --output ../generated/prisma`

The commits I made



Commits · mauricedb/cursorist		
github.com/mauricedb/cursorist/commits/building-the-...		
cursorist		
Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings		
Commits		
building-the-app		
All users All time		
Commits on Apr 29, 2026		
Apply this design to the /today and /inbox routes.	5d08327	<>
mauricedb committed 3 days ago		
Karpathy Behavioral Guidelines	4052d92	<>
mauricedb committed 3 days ago		
I want to be able to add new tasks.	8cdf164	<>
mauricedb committed 3 days ago		
instead of a standard HTML checkbox I want to use the Shadcn Checkbox and Label components.	be61d5c	<>
mauricedb committed 3 days ago		
There is a problem with marking a task as done. The task disappears from the list but the database is never updated. When I refresh the page I see the original list of task again while I was expect...	392ca57	<>
mauricedb committed 3 days ago		
Update the Prisma schema to store the tasks in the database. Use the current mock data to seed the database. Make the @app/(tasks)/today/page.tsx and @app/(tasks)/inbox/page.tsx pages use the data ...	2bedf32	<>
mauricedb committed 3 days ago		
On the @app/(tasks)/today/page.tsx route I would like an enhancement:	79c2ef4	<>
mauricedb committed 3 days ago		
Commits on Apr 28, 2026		
Basic list tasks UI	9fb6e61	<>
mauricedb committed 4 days ago		
Prisma rules	d0c3638	<>
mauricedb committed 4 days ago		
Build me a new landing page for a to-do application named Cursorist.	9eb9b41	<>
mauricedb committed 4 days ago		
Initialize Prisma ORM	ea65172	<>
mauricedb committed 4 days ago		
Initial commit from Create Next App	5e2034c	<>
mauricedb committed 4 days ago		



Intro to Cursor

Intro to Cursor

- The **AI-first code editor** — built on VS Code
- Not just autocomplete — **full codebase orchestration**
- Used by 2M+ developers to ship faster
- In this section you will learn:
 - What Cursor is and how it works
 - Its core features and how they fit together
 - How to set it up for Cursorist, our Next.js workshop application

What is Cursor?

- A code editor that puts **AI at the centre** — not as a plugin
- **Built on top of VS Code**
 - Same extensions, shortcuts, and settings you already know
 - Zero relearning curve
- Powered by **frontier models**
 - Claude, GPT-5.5, and others available
 - The model reads your entire codebase — not just the open file

The shift in how we code

- **Before — manual, file-by-file**
 - Write every line, search Stack Overflow, context in your head
- **With Cursor**
 - Tab-complete entire functions
 - Ask questions about your codebase in plain English
 - Refactor across 20 files with one prompt
 - Generate tests, find bugs, clean up tech debt on demand

Core features

- **Tab Completion**
 - predicts your next edit, whole functions at a time
- **Cursor Chat**
 - Ask questions about your code with full project context
- **Composer (Agent)**
 - Orchestrate multi-file changes from a single prompt
- **Cursor Rules**
 - Enforce your coding standards on every AI generation
- **Codebase Indexing**
 - Semantic search with @file, @folder, @web, @docs

Cursor Chat

- **Talk to your codebase** — not just an AI chatbot
- **Reference specific context using @ symbols**
 - @file — attach a specific file
 - @folder — entire directory
 - @web — live web search
 - @docs — library documentation
 - @codebase — semantic full-repo search
- **Example:** "*@UserTable.tsx Why does the sort break on empty arrays?*"

Composer (Agent mode)

- One prompt — multiple files changed simultaneously
- What you can do with Composer
 - Scaffold a full Next.js route (page, layout, loading, error)
 - Generate typed API routes and server actions end-to-end
 - Migrate class components to functional components with hooks
 - Add a feature across all affected files in one go
- You review the diffs — **you stay in control**

Cursor Rules

- A set of markdown files in your project
- The AI reads it on every Chat and Composer request
- What to put in them
 - Stack and framework instructions (Next.js 14, App Router)
 - Conventions: strict TypeScript, Tailwind, no inline styles
 - Rules: validate with Zod, no any types, use cn() for classes
- **Result: consistent, professional code** — every generation

Cursor + Next.js

- **Cursor understands the App Router file conventions**
 - Ask it to scaffold a route — page, layout, loading and error created automatically
- **Great for server actions and API routes**
 - Describe the endpoint, get typed code + Zod validation + Prisma query
- **UI iteration is instant**
 - Describe the component, get Tailwind-styled, accessible JSX
- **Works great with the Prisma schema**

The AI-native dev loop

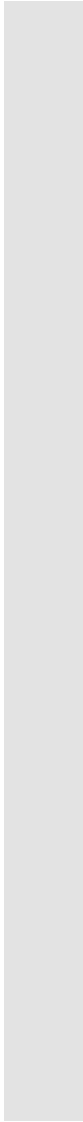
1. **Define**
 - Write Cursor Rules, tell the AI your stack
2. **Scaffold**
 - Use Composer to generate routes, schemas, components
3. **Iterate**
 - Tab completion + Chat to refine logic and UI
4. **Harden**
 - Generate tests, hunt edge cases, clean up tech debt
5. **Ship**
 - Review diffs, accept, commit. Repeat at 10x speed

Quick setup check

- **Before we start building**, make sure you have:
 - Cursor installed — cursor.com/download
 - Node.js 18+ installed
 - Git configured
 - Cursor account created (free tier is fine)
 - Workshop repo cloned — link in chat
 - Repo opened in Cursor — you should see AI suggestions
- **Raise your hand if anything is missing** — we build in 10 minutes!

Break time





Building the home page

A very naïve
prompt

“Build me a new landing page for a to-do application named Cursorist.”

The result



Cursorist - To-dos for focused teams

Organize every task. Ship work without the chaos.

Cursorist helps you capture tasks fast, prioritize the right work, and stay in flow from planning to done. Built for creators, operators, and product teams who move quickly.

[Start for free](#) [Explore features](#)

42% Faster task completion	3x Better weekly planning clarity	99.9% Task sync reliability
--------------------------------------	---	---------------------------------------

Capture instantly
Add tasks in seconds with smart defaults, keyboard shortcuts, and natural language dates.

Prioritize with confidence
Rank what matters using focus views, deadlines, and team-level workload balancing.

Finish in flow
Move from to-do to done with status pipelines, recurring tasks, and smart reminders that keep momentum high.

Ready to simplify your workday?
Join teams using Cursorist to plan better and execute faster.

[Create your workspace](#)

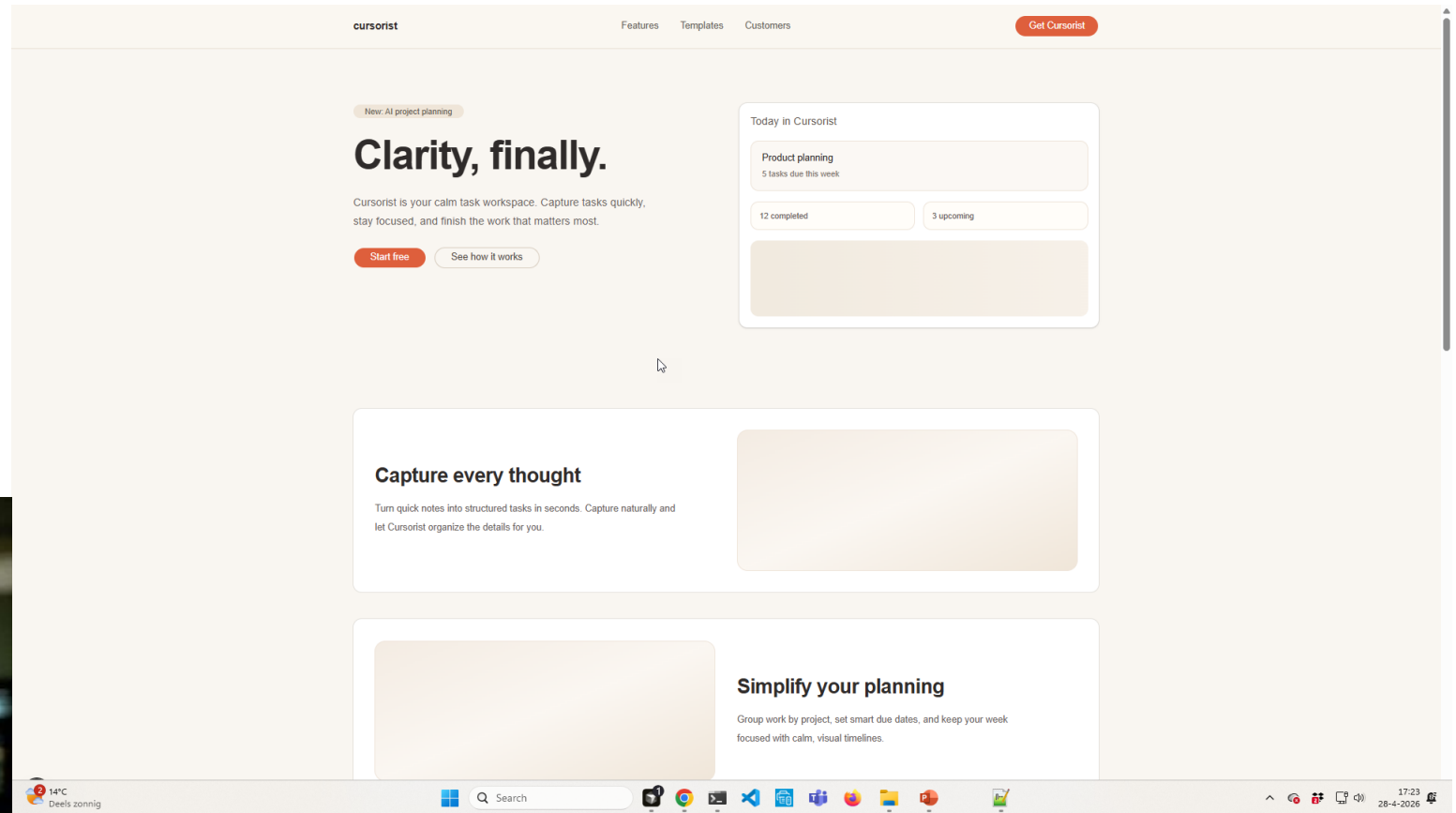
A slightly better prompt

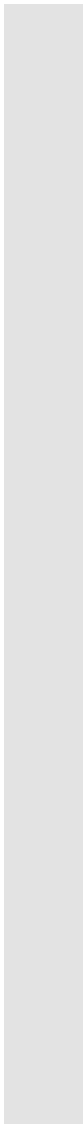
“Build me a new landing page for a to-do application named Cursorist.

Use the attached image as the design to follow exactly.

Use Shadcn components whenever they are available.”

The result





Cursor Rules

Product specific Cursor rules

- **Use Cursor Rule files** to give Cursor more guidance
- Include the following:
 - next-js.mdc
 - prisma.mdc
 - shadcn.mdc
 - tailwind.mdc

<https://github.com/sanjeed5/awesome-cursor-rules-mdc/tree/main/rules-mdc>

Application architecture Cursor rule

"/create-rule

I want you to create a rule about the general application architecture to adhere to.

Aks me questions and come up with suggestions to decide what the rule file should contain."

Application architecture



The screenshot displays the Cursor IDE interface. On the left, a chat window shows a conversation with an AI agent. The user asks for a rule to create a hybrid application architecture. The agent responds with a detailed rule and a code snippet. The main editor shows the generated rule, which is a YAML file named `architecture.mdc`. The rule is titled "Application Architecture" and describes a hybrid architecture with clear layer boundaries and safe data access patterns. It includes sections for "Target Structure" and "Required Boundaries". The "Target Structure" section lists folders for product areas (e.g., `billing`, `users`, `projects`) and logical layers (e.g., `ui`, `application`, `domain`, `infrastructure`). The "Required Boundaries" section lists rules for UI, server actions, and route handlers.

Chat History:

- User: I want you to create a rule about the natural configuration architecture.
- Agent: B) Only critical domain logic requires tests. C) No testing requirements in this rule. Once you answer, I'll generate the `.cursor/rules/<name>.mdc` file with clean frontmatter and concise, actionable architecture guidance.
- User: Perfect — I'll create a new always-applied architecture rule using your choices (hybrid style, balanced strictness, and boundaries A/B/C/D), then I'll share exactly what was added.
- User: Explored 1 file, 1 search. I've aligned with your existing rule style; next I'm adding a new `.cursor/rules` file for architecture with explicit "good/bad" boundaries and pragmatic exceptions.

Generated Rule (architecture.mdc):

```
---
description: Hybrid application architecture with clear layer boundaries and safe data access patterns.
alwaysApply: true
---
# Application Architecture

Use a hybrid architecture: organize by feature, and keep clear layers inside each feature.

## Target Structure

- Feature-first folders for product areas (for example `billing`, `users`, `projects`).
- Inside each feature, use logical layers:
  - `ui` - components, pages, presentation code
  - `application` - use-cases, orchestration, transactions
  - `domain` - business rules, entities, pure logic
  - `infrastructure` - Prisma repositories, external API adapters

## Required Boundaries

### 1) UI must not access the database directly

- Never import Prisma client in `app/**` or `components/**`.
- UI may call Server Actions, Route Handlers, or typed service functions only.

### 2) Server Actions and Route Handlers call services

- Keep handlers thin: parse input, call application service, return response.
```



My Cursor Workflow

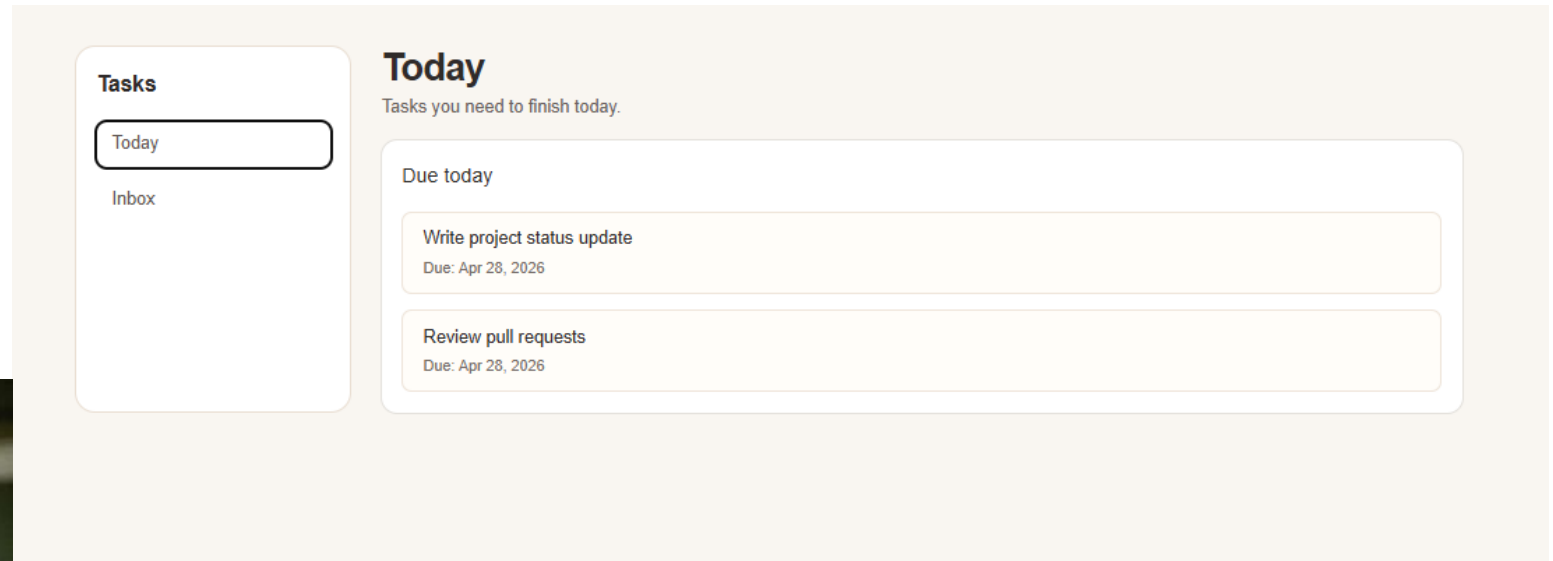
The first prompt

“Create a new page where I can see tasks I need to finish today under the path `/app/today`. Under the path `/app/inbox` I want to see a list of all tasks I need to complete.

I want a menu on the left of the screen to navigate between these views.

Just use a mock data structure for now.”

The result



Prompt to add details

“On the `@app/(tasks)/today/page.tsx` route I would like an enhancement:

- Also show task which are not done yet and past their due date.

On the `@app/(tasks)/inbox/page.tsx` and `@app/(tasks)/today/page.tsx` routes I would also like to see a number of enhancements:

- When a task is past the due date the line with "Due: {date}" should be shown in red.
 - Each item should have a checkbox to mark that task as done.
- Tasks which are not done and supposed to be done today should have their "Due: {date}" in green.”

The result



Tasks
Today
Inbox

Today
Tasks you need to finish today.

Due today and overdue

- ☐ Write product requirements document
Due: Apr 22, 2026
- ☐ Review pull requests
Due: Apr 29, 2026
- ☐ Write project status update
Due: Apr 29, 2026

Adding a database using Prisma ORM

“Update the Prisma schema to store the tasks in the database. Use the current mock data to seed the database. Make the `@app/(tasks)/today/page.tsx` and `@app/(tasks)/inbox/page.tsx` pages use the data from the database instead of the mock data they are using now.”

Seed and
inspect the
database



“npx prisma db seed”

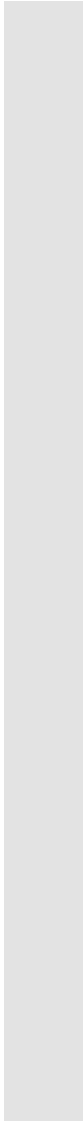
“npx prisma studio”

Fixing a bug

“There is a problem with marking a task as done. The task disappears from the list but the database is never updated. When I refresh the page I see the original list of task again while I was expecting not to see the done task.”

Break time





Using 3rd party libraries

Point Cursor at the documentation

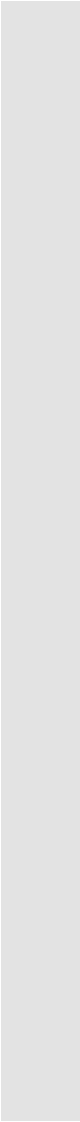
- The Cursor team has made a lot of **3rd party documentation** available
 - Sometimes better than letting the LLM just use more general knowledge
- Use the **@doc** directive to point at specific product docs when needed

The prompt



“instead of a standard HTML checkbox I want to use the Shadcn Checkbox and Label components.

Make sure to use the best practices
@Shadcn”



Cursor Plan Mode

Cursor Plan Mode

- **Think before you build.**
- Cursor has **four agent modes**:
 - **Agent** — the default; edits your code directly from a prompt
 - **Plan** — researches your codebase, asks clarifying questions, builds a step-by-step plan before touching a file
 - **Ask** — answers questions without making changes
 - **Debug** — hunts the cause of broken behaviour and suggests targeted fixes

Plan Mode in practice

- **Describe the feature**
 - Tell Cursor what you want to build
 - If you have a PRD then provide it
- **Answer its questions**
 - It asks clarifying questions to close gaps in requirements before writing a single line
- **Review the plan**
 - A Markdown file with file paths, code references, and a TODO checklist; edit it freely
- **Hit Build**
 - Cursor executes the plan step by step

Prompt to add new tasks

"I want to be able to add new tasks.

When enter the text date I want to be able to use a textual description to set the due date. For example:

- Add Playwright tests tomorrow
- Migrate to Supabase in two days
- Host the application on Vercel next week
- Add error logging next month

I also want to be able to just add the task title and select a due date manually using the Shadcn Date Picker component."

Cursor Plan Mode

The plan to build it

I want to be able to add new tasks.

When enter the text date I want to be able to use a manual picker to override the parsed date.

Perfect—thanks. I'll now produce a concrete implementation plan with those choices: broad natural-language parsing, and manual picker date overriding parsed text.

Read package.json L1-40

Read button.tsx L1-59

task_creation_with_due_da...

Task Creation With Due Dates

Add task creation with both natural-language due date parsing and manual Shadcn date picking, with manual date taking precedence when both are provided.

6 To-dos

Add create task use-case and repository insert function

Implement POST /api/tasks with validation, parse, and due-date precedence

Add natural-language due date parser utility returning date + cleaned title

... 3 more

View Plan

Auto

Build Ctrl+P

Plan, Build, / for commands, @ for context

Auto

task_creation_with_due_dates_2b3ce1f2.plan.md

Plans > task_creation_with_due_dates_2b3ce1f2.plan.md

Autosave Build Ctrl+P

Implement Task Creation With Parsed + Manual Due Dates

Goals

- Support adding tasks from the UI in both `Inbox` and `Today` pages.
- Parse broad natural-language due dates from the typed title (e.g. "tomorrow", "in two days", "next week", "next month").
- Allow selecting a due date manually through a Shadcn-style date picker; manual selection overrides parsed date.

Architecture Changes

- Keep UI thin: form submits to a route handler; route handler validates/parses input and calls application service.
- Keep application/service layer responsible for orchestration only.
- Keep repository layer responsible for Prisma writes and read mapping.

Implementation Steps

1. Add create-task backend flow
 - Extend application service in `c:\Repos\cursorist\features\tasks\application\task-service.ts` with a `createTask` use-case.
 - Extend repository in `c:\Repos\cursorist\features\tasks\infrastructure\prisma-task-repository.ts` with `createTaskInDb` that writes `id`, `title`, `dueDate`, `completed=false`.
2. Add POST API endpoint for task creation
 - Add `c:\Repos\cursorist\app\api\tasks\route.ts` for `POST`.
 - Validate request shape (`title`, optional `manualDueDate`), parse natural-language date from title, resolve final due date with precedence:
 - manual picker date (if provided)
 - parsed natural-language date (if detected)
 - otherwise validation error (or fallback, see note below)
 - Return created task payload for optimistic UI update.
3. Implement natural-language date parsing utility
 - Add utility module under tasks feature (e.g. `c:\Repos\cursorist\features\tasks\domain\due-date-parser.ts`).
 - Use a date parsing library (broad NL support) to extract date from title text, normalize to UTC date boundary used by existing task logic.
 - Also return cleaned title (title with detected date phrase removed) so created tasks are stored without trailing temporal phrases.
4. Add Shadcn-style date picker UI primitives
 - Add required UI components in `components/ui` (calendar/popover/button pattern) and any missing dependencies.
 - Ensure the picker exposes a nullable selected date so users can rely only on text parsing when desired.
5. Add task creation form component

CURSORIST

.cursor

.next

.vscode

app

components

features

generated

lib

node_modules

prisma

public

.env

.gitignore

AGENTS.md

CLAUDE.md

components.json

dev.db

eslint.config.mjs

next-env.d.ts

next.config.ts

package-lock.json

package.json

pnpm-lock.yaml

postcss.config.mjs

prisma.config.ts

README.md

tsconfig.json

tsconfig.buildinfo

OUTLINE

TIMELINE

IDENTIFY

Cursor Tab

Reviewing

No Environment

Prettier

The result



Tasks

Today

Inbox

Inbox

All tasks you need to complete.

All open tasks

Pick due date (optional)

Add task

☐ Migrate to Supabase
Due: May 1, 2026

☐ Plan sprint backlog
Due: Apr 30, 2026

☐ Refactor onboarding form
Due: May 6, 2026

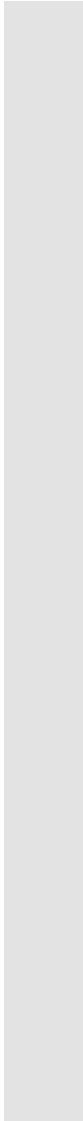
Break time





Karpathy Behavioral Guidelines

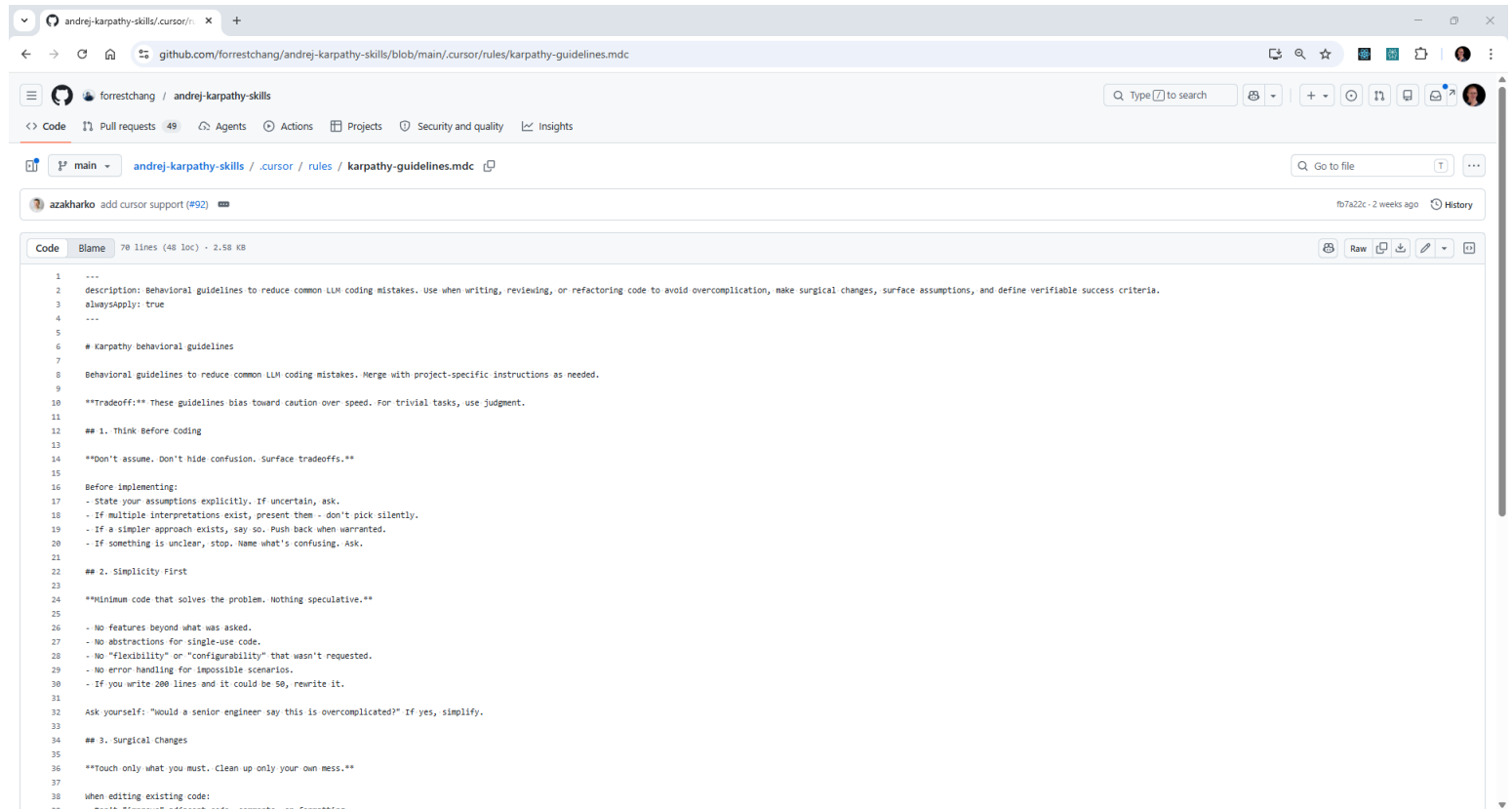
Andrej Karpathy is an AI researcher and co-founder of OpenAI



Karpathy Behavioral Guidelines

- **Models make wrong assumptions and run with them**
 - Always give explicit context and constraints upfront rather than leaving gaps for them to fill
- **Models don't surface confusion, inconsistencies, or trade-offs**
 - You have to actively prompt for pushback and alternative approaches
- **Models overcomplicate and bloat**
 - Code, abstractions, and APIs all need to be audited; always ask "could this be simpler?"
- **Models don't clean up after themselves**
 - Dead code, stale comments, and unrelated side-effect changes accumulate and need manual review
- **Sycophancy means Models agree with your bad ideas**
 - Treat their confirmation as a starting point, not validation
- **Shift from imperative to declarative prompting**
 - Give success criteria and let them loop, rather than dictating step-by-step what to do

The Karpathy Cursor Rule



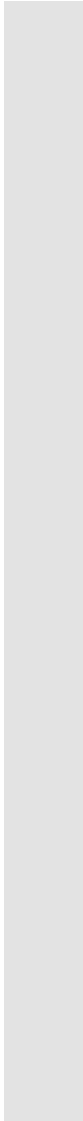
The screenshot shows a web browser displaying a GitHub repository page for 'andrej-karpathy-skills'. The file 'karpathy-guidelines.mdc' is open, showing a document with behavioral guidelines for coding. The document is structured with a description, a list of guidelines, and a section for surgical changes.

```
1 ---
2 description: Behavioral guidelines to reduce common LLM coding mistakes. Use when writing, reviewing, or refactoring code to avoid overcomplication, make surgical changes, surface assumptions, and define verifiable success criteria.
3 alwaysApply: true
4 ---
5
6 # Karpathy behavioral guidelines
7
8 Behavioral guidelines to reduce common LLM coding mistakes. Merge with project-specific instructions as needed.
9
10 **tradeoff:** These guidelines bias toward caution over speed. For trivial tasks, use judgment.
11
12 ## 1. Think Before coding
13
14 **Don't assume. Don't hide confusion. Surface tradeoffs.**
15
16 Before implementing:
17 - State your assumptions explicitly. If uncertain, ask.
18 - If multiple interpretations exist, present them - don't pick silently.
19 - If a simpler approach exists, say so. Push back when warranted.
20 - If something is unclear, stop. Name what's confusing. Ask.
21
22 ## 2. Simplicity First
23
24 **Minimum code that solves the problem. Nothing speculative.**
25
26 - No features beyond what was asked.
27 - No abstractions for single-use code.
28 - No "flexibility" or "configurability" that wasn't requested.
29 - No error handling for impossible scenarios.
30 - If you write 200 lines and it could be 50, rewrite it.
31
32 Ask yourself: "Would a senior engineer say this is overcomplicated?" If yes, simplify.
33
34 ## 3. Surgical Changes
35
36 **Touch only what you must. Clean up only your own mess.**
37
38 When editing existing code:
39     - Don't "improve" unrelated code / remove an annotation
```

The Karpathy Cursor Rule



- **Four behaviours the rule enforces on every agent session:**
 - **Think before coding**
 - Surface assumptions and ask clarifying questions before writing a line
 - **Simplicity first**
 - No speculative features, no abstractions for single-use code; if 200 lines could be 50, rewrite it
 - **Surgical changes**
 - Touch only what the request requires; mention unrelated issues, don't silently fix them
 - **Goal-driven execution**
 - Define verifiable success criteria upfront so the agent can loop independently
- *The link on the previous slide has the complete rule file — copy it into `.cursor/rules/karpathy-guidelines.mdc` and these behaviours apply to every session automatically*



Implementing a Google Stitch Design

Download the Stitch design

```## Stitch Instructions

Get the images and code for the following Stitch project's screens:

Project

Title: Todoist Redesign Dashboard

ID: 13452193057897107397

Screens:

1. Todoist Redesign Dashboard

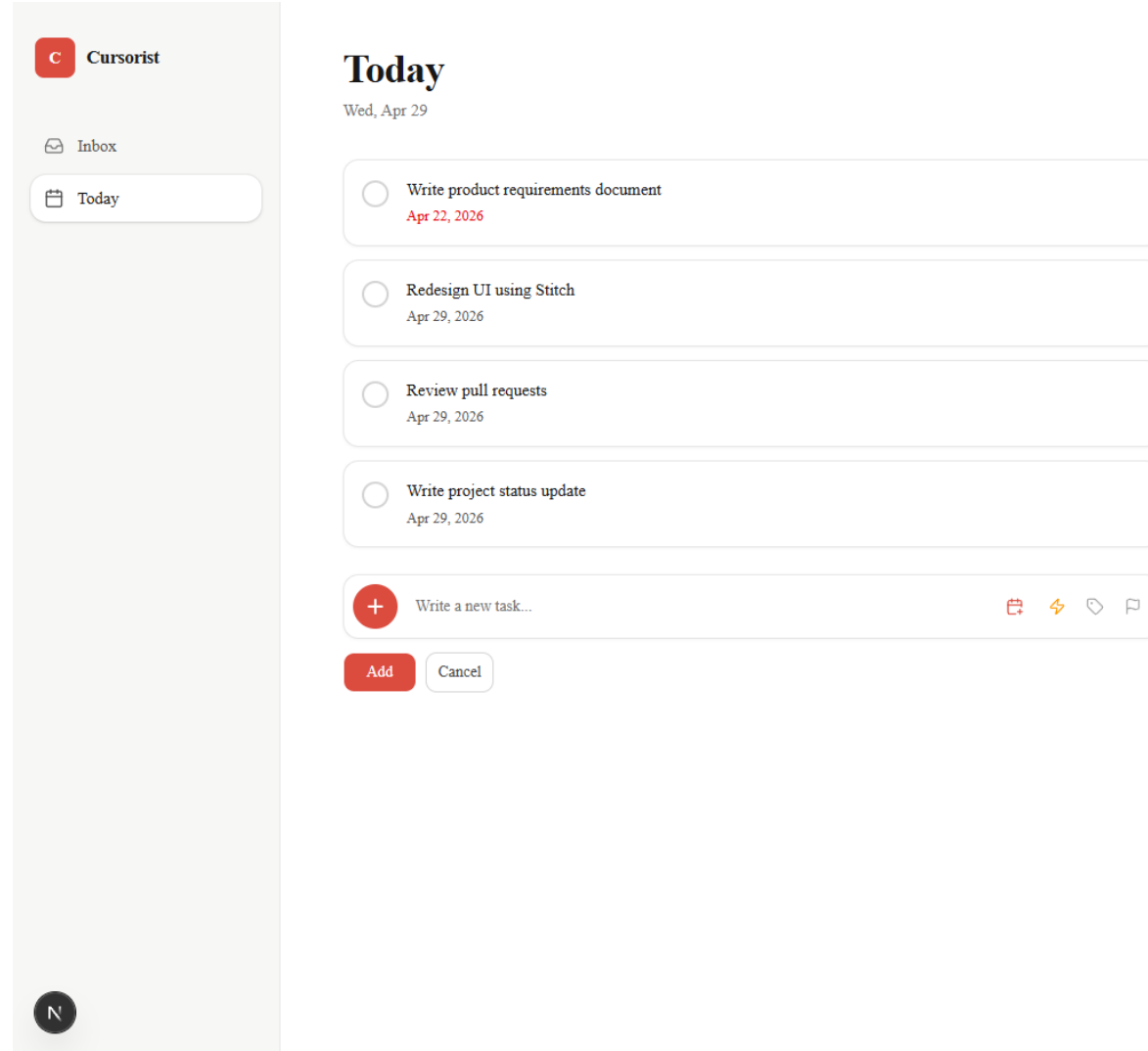
ID: afd7370692174a90ao8b7dd8bof1ab8o

Use a utility like `curl -L` to download the hosted URLs."

Implement the
Stitch design

“Apply this design to the
/today and /inbox routes.”

The result





Best Practices with Cursor

Best Practices with Cursor

- **Set the stage first**
 - Write Cursor Rules before you build, including an architecture rule for your specific app
- **One session, one feature**
 - Start a new Composer session per feature; stay in it for tweaks and refinements
- **Plan before you build**
 - Use plan mode for complex features and review the plan before accepting it
- **Declare outcomes, not steps**
 - Tell Cursor what you want to achieve, not how to do it
- **Always have an escape hatch**
 - Commit to Git before every agent run so you can revert if it goes off the rails

Context is everything

"An agent with a well-scoped context of 4–5 relevant files almost always outperforms one given 30 files 'just in case.'"

Why? More context isn't clarity — it's noise. The model has to guess what matters. You already know. So tell it.

Treat agent runs like deployments

"Commit first, review diffs carefully, and give the agent a concrete exit condition — a passing test, a specific behaviour — rather than an open-ended 'make it work' prompt."

Why? Open-ended runs drift. A clear finish line keeps the agent focused and makes the diff reviewable in under a minute.

Schema-first for full-stack work

"Define your shared types and API contracts before asking Composer to implement either side."

Why? When both the frontend and backend sessions reference the same schema file, type drift essentially disappears. The schema is the single source of truth — write it first.

Smaller Prompts, more sessions

"Shorter, focused sessions with a clear acceptance criterion produce cleaner, more reviewable diffs — especially once your codebase grows past a few hundred files."

Why? A giant prompt is a wish. A focused prompt with a finish line is an instruction. The agent performs better, and you stay in control of what changed and why.

Thank you for joining

[Share your thoughts](#)

